

Parallel and distributed Computing

Lecture 17

Bazam-e-Jahan

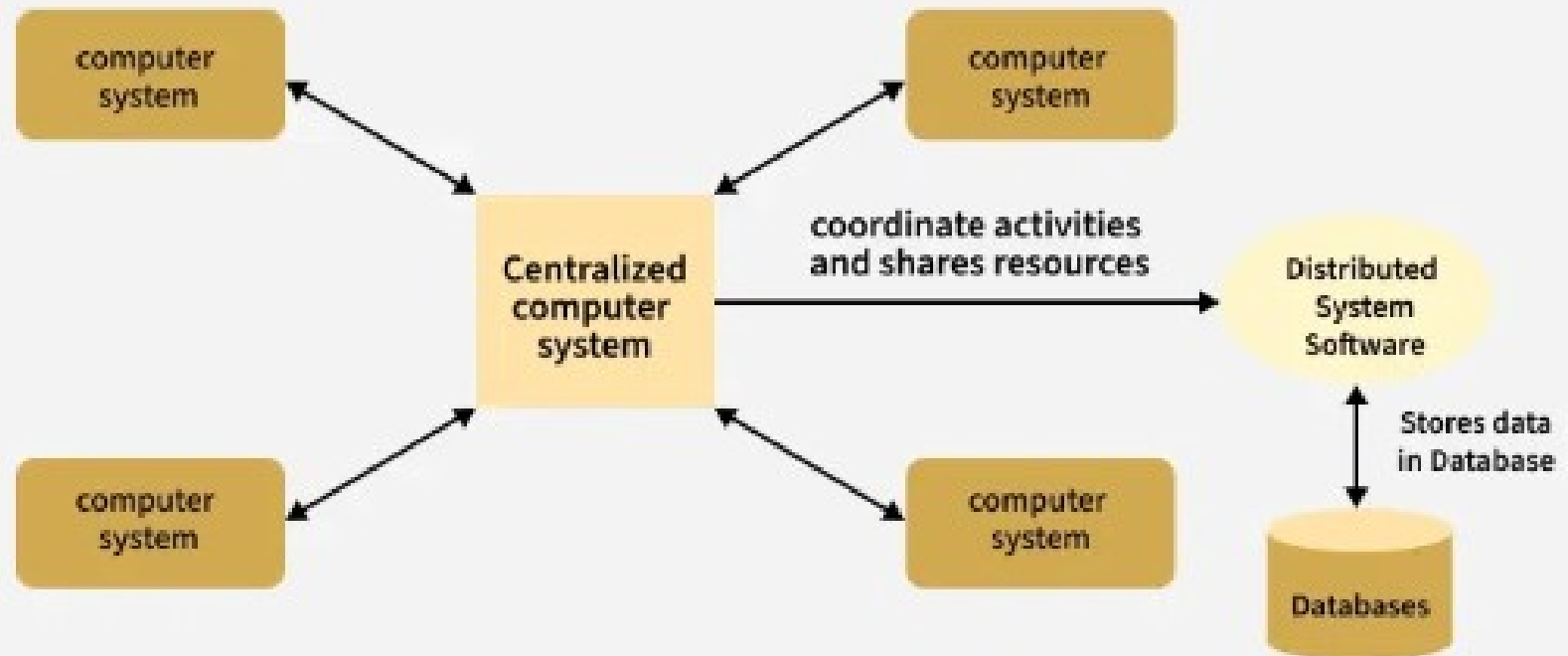
Lecturer

The University of Faisalabad

Memory consistency model

- In distributed systems, consistency models establish criteria for data synchronization and specify how users and applications should interpret data changes across several nodes.
- In a distributed system, it specifically controls how data is accessed and changed across numerous nodes and how clients are informed of these updates.

What are Consistency models in distributed systems?



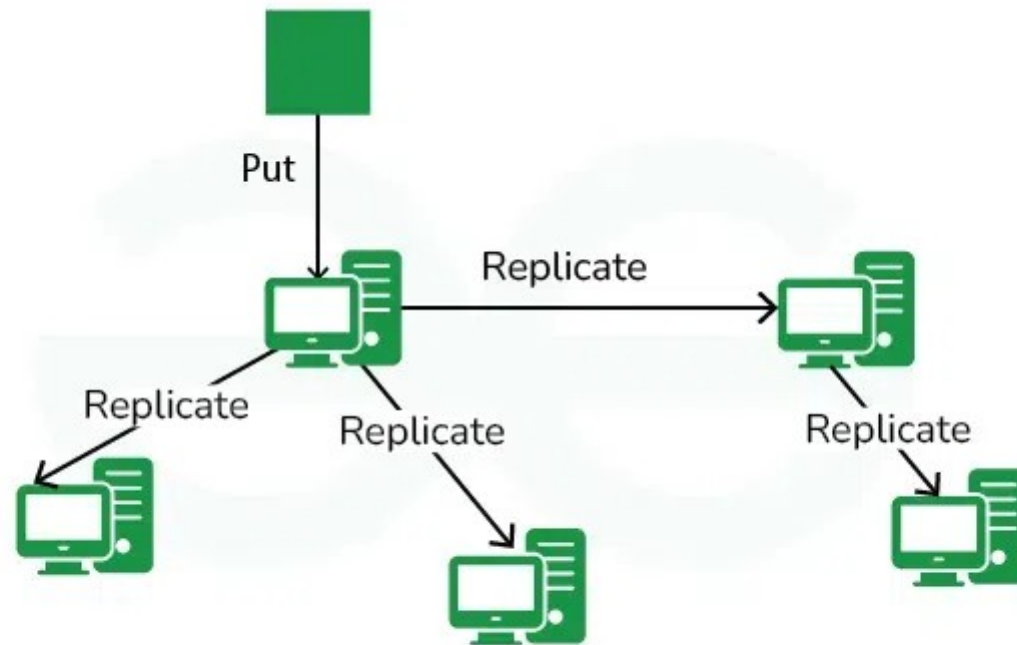
Consistency Model in Distributed System

Types of Consistency Models



- Strong Consistency
- Sequential Consistency Model
- Causal Consistency Model
- Weak Consistency Model
- Session Consistency
- Monotonic Reads and Writes

- In a strongly consistent system, all nodes in the system agree on the order in which operations occurred. Reads will always return the most recent version of the data, when an update occurs on one server, this model makes sure every other server in the system reflects this change immediately. This model provides the highest level of consistency, but it can be slower and require more resources in a distributed environment



Strong Consistency

Sequential Consistency Model

- It is a consistency model in distributed systems that ensures all operations across processes appear in a single, unified order. In this model, every read and write operation from any process appears to happen in sequence, regardless of where it occurs in the system. Importantly, all processes observe this same sequence of operations, maintaining a sense of consistency and order across the system.

Causal Consistency Model

- The Causal Consistency Model is a type of consistency in distributed systems that ensures that related events happen in a logical order. In simpler terms, if two operations are causally related (like one action causing another), the system will make sure they are seen in that order by all users. However, if there's no clear relationship between two operations, the system doesn't enforce an order, meaning different users might see the operations in different sequences.

Weak Consistency Model

- A weakly consistent system provides no guarantees about the ordering of operations or the state of the data at any given time. Clients may see different versions of the data depending on which node they connect to. This model provides the highest availability and scalability but at the cost of consistency.

- Session Consistency guarantees that all of the data and actions a user engages with within a single session remain consistent. Consider it similar to online shopping: session consistency ensures that an item will always be in your cart until you check out or log out, regardless of how you explore the page.
- In a distributed system, this means if you're interacting with different servers or services, you'll always get the same view of your data during your session.
- It might not reflect the most recent updates from other users, but for your own actions, it will remain consistent and reliable.

Monotonic Reads and Writes

- Once a piece of data has been read or written, monotonic reads and writes guarantee that the data will always be viewed in a predictable and consistent order in subsequent reads or writes.
- **Monotonic Reads:** If you read a value from a system, the next time you read it, you will either get the same value or a more recent one. You won't get an older value after seeing a newer one.
- **Monotonic Writes:** This ensures that once a write happens, all future writes will follow in the correct order. If you update a record or send a message, the system guarantees that it won't reverse the order of your updates.

Kindly read the book before and after the lecture, if some terms/ words are not covered in the lecture, if appear in exam it might not appear strange or out of course.